Enhancing LLM Fine-tuning for Text-to-SQLs by SQL Quality Measurement

Shouvon Sarker, Xishuang Dong, Xiangfang Li, Lijun Qian
CREDIT Center and Department of Electrical and Computer Engineering
Prairie View A&M University
Prairie View, TX 77446, USA
Email: ssarker3@pvamu.edu, xidong@pvamu.edu, xili@pvamu.edu, liqian@pvamu.edu

***Abstract*—Text-to-SQLs enables non-expert users to effortlessly retrieve desired information from relational databases using natural language queries. While recent advancements, particularly with Large Language Models (LLMs) like GPT and T5, have shown impressive performance on large-scale benchmarks such as BIRD, current state-of-the-art (SOTA) LLM-based Text-to-SQLs models often require significant efforts to develop auxiliary tools like SQL classifiers to achieve high performance. This paper proposed a novel approach that only needs SQL Quality Measurement to enhance LLMs-based Text-to-SQLs performance. It establishes a SQL quality evaluation mechanism to assess the generated SQL queries against predefined criteria and actual database responses. This feedback loop enables continuous learning and refinement of model outputs based on both syntactic correctness and semantic accuracy. The proposed method undergoes comprehensive validation on the BIRD benchmark, assessing Execution Accuracy (EX) and Valid Efficiency Score (VES) across various Text-to-SQLs difficulty levels. Experimental results reveal competitive performance in both EX and VES compared to SOTA models like GPT4 and T5.**

Keywords–Text-to-SQLs; Large Language Models (LLMs); SQL Quality Measurement; Database Information Retrieval

I. Introduction

Text-to-SQLs [1], [2], [3], [4] aims to convert users' queries into executable and structured SQL statements, where these queries are presented in natural languages. This technique enables non-expert users, particularly those are not familiar with the SQL language, to seamless access to knowledge stored in rational databases for applications in various areas such as finance and medicine. Nonetheless, it poses greater challenges compared to general semantic parsing tasks [2], [4]. First, even simple user queries may involve complex combinations of multiple tables and filtering requirements, necessitating sophisticated context understanding techniques to fulfill query needs. Secondly, there remains a significant gap between existing Text-to-SQL methods and real-world applications in efficiently and effectively generating high-quality SQLs, especially when dealing with large-scale databases.

Deep learning technologies have greatly advanced Text-to-SQLs methods through developing both non-seq2seq methods and seq2seq methods classified in terms of model architectures [2], [1]. For non-seq2seq methods [5], [6], [7], they typically involve two stages: 1) it employ encoder models with attention mechanisms like BERT [8] to learn high-quality representations of user queries; 2) it constructs sketch-based or grammar-based systems to generate SQL statements. On the other hand, seq2seq methods [9], [10], [11] treat Text-to-SQLs as a machine translation task, directly translating user queries into SQL statements in an end-to-end manner. This approach has achieved competitive performance through fine-tuning with minimal effort. However, a reliable Text-to-SQLs parser has yet to be fully implemented.

The emergence of Large Language Models (LLMs) has recently facilitated Text-to-SQL methods by building an interface between non-expert users and relational databases, leveraging the robust linguistic and coding capabilities of LLMs. This has sparked a new trend of *LLM-based Text-to-SQL approaches* [3]. Specifically, the in-context learning and domain generalization abilities of LLMs have significantly improved Text-to-SQL performance. For example, DIN-SQL decomposes the Text-to-SQL task into smaller sub-tasks to enhance performance on datasets like Spider [12]. DAIL-SQL explores the selection and organization of helpful examples in prompts in few-shot scenarios through supervised fine-tuning and a systematic study of in-context learning [13]. ACT-SQL introduces a Chain-of-Thought (CoT) [14] prompt to enhance reasoning abilities in SQL generation, extending to multi-turn Text-to-SQL tasks [15]. Furthermore, to address database schema with few rows of values, the Big bench for large-scale Database (BIRD) is proposed to bridge the gap between academic study and real-world applications [4]. Text-to-SQL methods applied on BIRD have garnered significant attention [4], [16], [17] , achieving high performance through additional efforts such as developing SQL classifiers [4], fine-tuning LLMs [16], or facilitating multi-agent LLM collaborations [17]. Although these research efforts have advanced LLM-based Text-to-SQL systems, they often require complex data preprocessing or data augmentation techniques.

In this study, we aim to enhance LLM-based Text-to-SQL by using straightforward feedback mechanisms and human-designed step-by-step prompts (HDSP). The method requires only LLMs reasoning and low-cost SQL quality assessment. The flow of the proposed method is illustrated in Figure 1. First, in the prompt engineering block, we design step-by-step prompts to LLMs for enhancing the Text-to-SQL performance. Next, we implement a feedback mechanism through SQL quality measurement. This mechanism compares generated SQL statements against expected outcomes, providing direct feedback on SQL difficulties to automatically refine the prompts. We systematically validate our approach on BIRD datasets, assessing Execution Accuracy (EX) and Valid Efficiency Score (VES) [4]. Experimental results demonstrate

2375-9259/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDMW69685.2025.00343

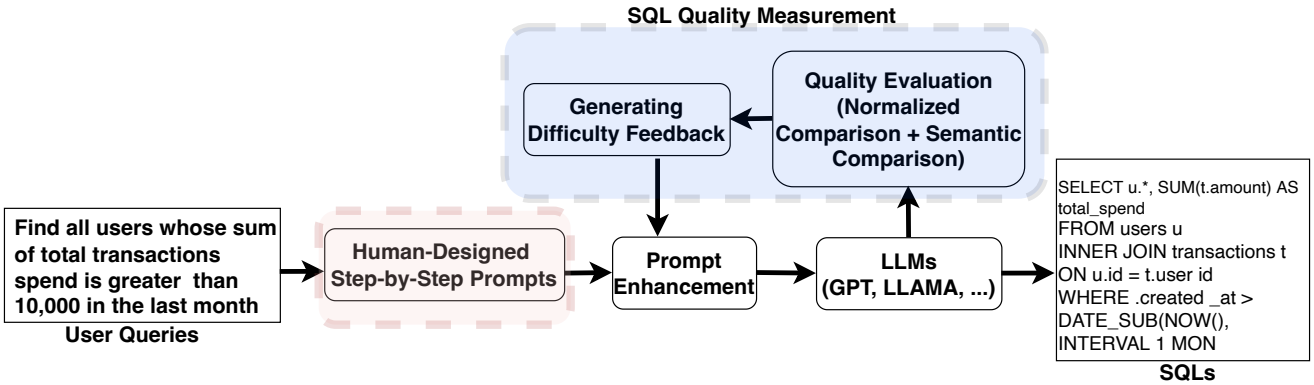


Figure 1: Flow of Enhancing LLM Fine-tuning for Text-to-SQLs by SQL Quality Measurement.

significant improvements in both metrics compared to state-of-the-art models like GPT4 [18].

In summary, our contributions are as follows:

- We design step-by-step prompts that provide detailed instructions to LLMs to complete the Text-to-SQLs task effectively.
- We implement SQL statement quality feedback by comparing generated SQL statements against expected outcomes, both normally and semantically, to inform automatic prompt refinement.
- Extensive experiments are conducted to validate the proposed method, assessing accuracy and efficiency. Experimental results confirm that our approach not only automates prompt construction but also enhances prompts based on SQL statement quality, thereby improving Text-to-SQL performance.

II. Task Formulation

LLMs-based Text-to-SQL involves utilizing LLMs to convert natural language user queries into executable SQL statements. Let Q represent a natural language query and D denote the database schema, where $D = (T, C)$ with $T = \{t_1, ..., t_m\}$ representing multiple tables and $C = \{c_1, ..., c_n\}$ representing columns. The objective is to generate a SQL statement $\hat{Y}$ that is executable on D to retrieve the information required by Q. Formally, given a prompt template $P(Q, D)$ and external knowledge evidence K, the generation process of the SQL statement $\hat{Y}$ by a large language model (LLM) M can be defined as a conditional probability distribution:

$$P_M(\hat{Y}|P(Q,D)) = \prod_{i=1}^{|\hat{Y}|} P_M(\hat{Y}_i|\theta, P(Q,D), K, \hat{Y}_{1:i-1}) \quad (1)$$

It represents the likelihood of generating SQL statement $\hat{Y}$ given the natural language query Q, the database schema D, any external knowledge evidence K, and the prompt template P. The LLM generates each token, denoted by $\hat{Y}_i$ representing the i-th token of the SQL statement $\hat{Y}$. $|\hat{Y}|$ denotes the length of the SQL statement $\hat{Y}$, and θ represents the parameter of the LLM. Notably, $P_M(\hat{Y}|P(Q,D))$ is assessed in terms of syntactic correctness and semantic accuracy, defined as execution accuracy.

III. Methodology

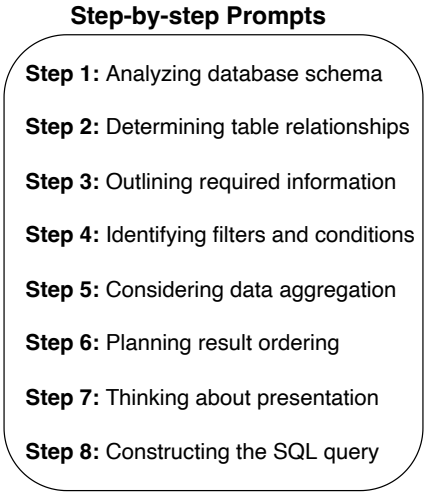


Figure 2: Human-designed step-by-step (HDSP) prompts for Text-to-SQLs via LLMs.

As shown in Figure 1, the proposed method include two special components, namely, human-designed step-by-step prompts and SQL quality measurement.

A. Step-by-step Prompts

Figure 2 presents details of the step-by-step prompts for Text-to-SQLs via LLMs. It include eight steps as below.

1) LLMs are encouraged to begin by understanding the database schema. This involves identifying the available tables and their relevant columns for the user query, which is crucial for generating accurate SQL statements.

2) LLMs are prompted to identify how tables are interconnected by determining the types of joins needed to connect them, essential for SQL statements involving multiple tables.
3) LLMs consider the specific information to extract from each table, including which columns to select and any necessary calculations.
4) LLMs pinpoint any required filters or conditions to apply to the data, such as `WHERE` clauses, to refine the SQL statements according to the query's requirements.
5) LLMs assess whether the SQL statements involve aggregating data, potentially using `GROUP BY` or `HAVING` clauses to summarize or filter aggregated data.
6) LLMs determine how to order the SQL statements, which is crucial where the order of the SQL statements matters (e.g., the top 10 results).
7) LLMs consider the final presentation of the SQL statements, deciding whether to use subqueries for clarity or `WITH` clauses to simplify complex queries.
8) LLMs synthesize the information gathered in the previous steps into coherent SQL statements.

These step-by-step prompts are able to ensure that the SQL statements accurately reflect the natural language requirements of the user query, where LLMs can adjust their strategies, and gradually improve SQL generation capabilities.

B. SQL Quality Measurement

As shown in Figure 1, it needs the quality of generated SQL to build a feedback of SQL difficulties to refine the prompts. Measuring the SQL quality includes two components: Normalized Comparison and Semantic Comparison.

1) Normalized Comparison: It quantifies the syntactic similarity between the SQL statement $\hat{Y}$ and the ground truth Y. Levenshtein distance ($L(\cdot)$) is employed as the metric for evaluating their similarity, defined as the minimum number of single-character edits required to change one string into another. Thus, the normalized comparison can be defined as:

$$f_{nc}(\hat{Y}, Y) = 1 - \frac{L(f_{Norm}(\hat{Y}), f_{Norm}(Y))}{\max(\left|f_{Norm}(\hat{Y})\right|, |f_{Norm}(Y)|)} \quad (2)$$

where $f_{Norm}(\cdot)$ represents the normalization of an input.

2) Semantic Comparison: Semantic comparison is defined as a binary indicator of whether the type of the answer $\hat{A}$ generated through running the generated SQL statement ($\hat{Y}$) is same to that of the answer (A') generated by ChatGPT directly in terms of the user query, which is defined as:

$$f_{sc}(\hat{A}, A') = \begin{cases} 1 & \text{if } R(\hat{A}) = R(A') \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

where $R(\cdot)$ is to check the type of the input.

In terms of the results of the normalized comparison, the feedback of SQL difficulties is defined by

$$f_{diff}(f_{nc}, N_{col}) = \begin{cases} 0 & if \;\; f_{nc} < t_{nc} \;\; and \;\; N_{col} \leq 5 \\ 1 & if \;\; f_{nc} < t_{nc} \;\; and \;\; 5 < N_{col} < 10 \\ 2 & if \;\; f_{nc} < t_{nc} \;\; and \;\; N_{col} \geq 10 \end{cases} \quad (4)$$

where 0, 1, and 2 denote simple, moderate, and difficult SQL statements, t_{nc} denotes a threshold of the normalized comparison, and N_{col} refers to the number of columns involved in the SQL statement. Furthermore, a specific feedback to implement the prompt enhancement shown in Figure 1 is defined by

$$f_{sf}(f_{nc}, f_{sc}, f_{diff}) = \begin{cases} 1 & if \;\; f_{nc} \geq t_{nc} \;\; and \;\; f_{sc} = 0 \\ & or \;\; f_{nc} < t_{nc} \;\; and \;\; f_{diff} = 1 \\ & or \;\; f_{nc} < t_{nc} \;\; and \;\; f_{diff} = 2 \\ 0 & otherwise \end{cases} \quad (5)$$

If the value of f_{sf} is 1, the step-by-step prompts are extended with five steps below.

1) Complex Joins and Subqueries: LLMs are encouraged to plan the ordering of results and the combination of multiple datasets. This involves deciding on the appropriate `JOIN` types and conditions.
2) Data Transformation and Calculations: LLMs are prompted to identify any complex calculations or transformations needed for the data. This includes creating new columns based on existing data using SQL functions or handling `date` and `time` calculations.
3) Optimization: LLMs review the query for performance and optimization, considering indexing and query simplification to improve execution speed.
4) Security and Data Integrity: LLMs ensure that the query adheres to security practices and maintains data integrity, avoiding SQL injection risks by using parameterized queries or prepared statements.
5) Review and Test: Before finalizing, review the query thoroughly. Test it against different scenarios to ensure it covers all edge cases and produces expected results.

These five steps are inserted into the step-by-step prompts between step five and step six shown in Figure 2 to further enhance Text-to-SQLs. The flow of the proposed feedback mechanism is summarized as Figure 3.

IV. Experiments

A. Dataset

BIRD (**BI**g Bench for La**R**ge-scale **D**atabase Grounded Text-to-SQL Evaluation)[1] is a pioneering, cross-domain dataset designed to examine the impact of extensive database contents on text-to-SQL parsing [4]. BIRD includes over 12,751 unique question-SQL pairs and 95 large databases with a total size of 33.4 GB. It spans more than 37 professional domains, such as blockchain, hockey, healthcare, and education. Designed

[1]https://bird-bench.github.io

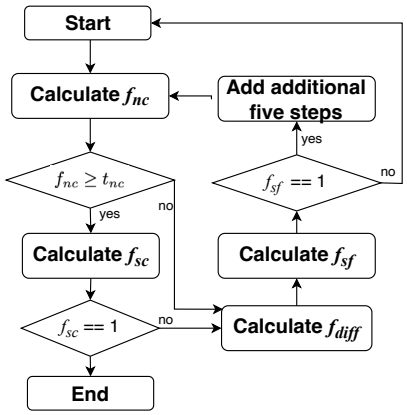


Figure 3: Flow of the proposed feedback mechanism.

to address the complexities of working with large, real-world databases, BIRD emphasizes the importance of understanding database values, incorporating external knowledge for accurate SQL query generation, and focusing on SQL execution efficiency. BIRD not only presents new challenges for developing more effective text-to-SQL models but also sets a new benchmark for evaluating model performance in real-world scenarios. It highlights the gap between current machine learning models and human-level SQL query formulation. Figure 4 presents one example of data sample from BIRD benchmark.

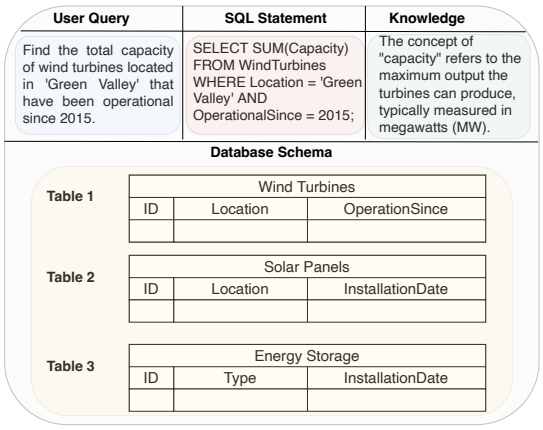


Figure 4: An example of data sample from BIRD benchmark.

B. Experiment Setup

We utilized a range of large language models (LLMs), such as T5 and GPT variants, each configured in unique setups to investigate various enhancement strategies, including the integration of external knowledge, prompt engineering, and feedback mechanisms. Each LLM was fine-tuned with a dataset tailored specifically to Text-to-SQL tasks, ensuring familiarity with the relevant vocabulary and structure. The models were tested under controlled conditions to precisely measure their performance in generating SQL statements.

C. Evaluation Metrics

To assess the performance of LLMs in generating SQL statements, two primary metrics were employed [4]:

- Execution Accuracy (EX): This metric quantifies the correctness of the generated SQL statements by comparing them to a set of predefined ground truths. A higher EX score indicates a higher similarity between the generated SQL statements and the ground truth, reflecting the model's understanding and query generation capabilities.
- Valid Efficiency Score (VES): VES evaluates the efficiency of the SQL statements generated by the models, taking into account both the accuracy of the SQL statement and its execution time against the database. A higher VES signifies not only correct but also optimally efficient generation of SQL statements, which is crucial for real-time applications of autonomous systems.

These metrics offer a comprehensive view of the models' performance, highlighting not just the accuracy but also the efficiency of the generated SQL statements, two critical aspects in the deployment of autonomous systems across various sectors.

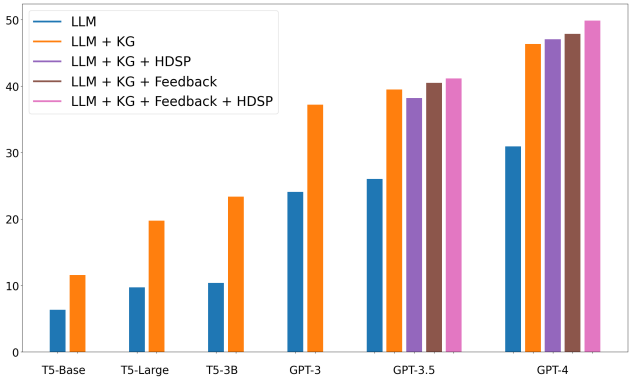
D. Result and Discussion

Table I presents a performance comparison between baseline models (T5 and GPT-3) and proposed methods based on GPT-3.5 and GPT-4, using EX and VES scores. Overall, incorporating knowledge (KG) significantly enhances the performance of LLM-based Text-to-SQL models. Additionally, when comparing the performance of LLMs using human-designed step-by-step prompts (HDSP) and feedback mechanisms, the feedback mechanism proves to be more beneficial. Furthermore, combining HDSP with the feedback mechanism allows GPT-4 to achieve the highest EX values, indicating that feedback effectively enhances the prompts within the HDSP framework shown in Figure 1. However, this combination reduces the VES values due to increased complexity in executing SQL statements, particularly with the feedback mechanism.

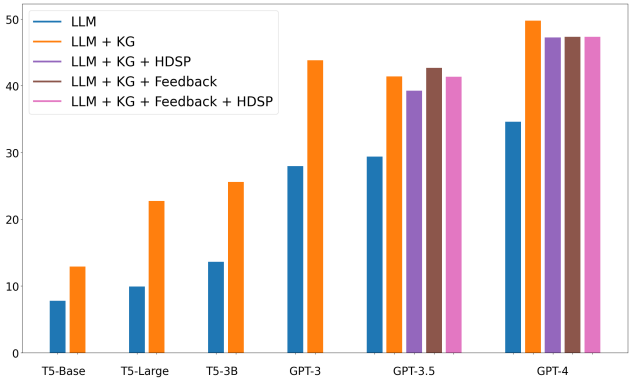
TABLE I: Comparing baselines with proposed methods by EX and VES scores, where KG and HDSP denote knowledge and human-designed step-by-step prompts, respectively.

LLMs		EX	VES
T5	T5-Base	6.32	7.78
	T5-Large	9.71	9.90
	T5-3B	10.37	13.62
	T5-Base + KG	11.54	12.90
	T5-Large + KG	19.75	22.74
	T5-3B + KG	23.34	25.57
GPT-3	GPT-3	24.05	27.97
	GPT-3 + KG	37.22	43.81
GPT-3.5	GPT-3.5	26.01	29.39
	GPT-3.5 + KG	39.48	41.39
	GPT 3.5 + KG + HDSP	38.20	39.25
	GPT-3.5 + KG + Feedback	40.48	42.68
	GPT-3.5 + KG + Feedback + HDSP	41.16	41.37
GPT-4	GPT-4	30.90	34.60
	GPT-4 + KG	46.35	**49.77**
	GPT-4 + KG + HDSP	47.06	47.26
	GPT-4 + KG + Feedback	47.86	47.35
	GPT-4 + KG + Feedback + HDSP	**49.87**	47.32

Furthermore, Figure 5 provides a clear visualization of the performance across different LLM configurations, making it easier to recognize performance trends. For EX scores, GPT-4 outperformed other LLMs when combined with HDSP and feedback. However, for VES scores, it was observed that the use of feedback and HDSP actually reduced performance.



(a) EX scores across different LLMs configurations.



(b) VES scores across different LLMs configurations.

Figure 5: A bar chart provides a clear visualization of the performance of different LLMs configurations.

Since GPT models outperform T5, we focus on GPT-based Text-to-SQL performance across different sample types. Tables II and III show the performance of GPT models on three sample types as predefined in the BIRD datasets. Incorporating knowledge (KG) enhances performance across all sample types, indicating that additional information about the database, such as schema and column details, aids in generating SQL statements. The feedback mechanism is particularly beneficial for improving performance on simple samples compared to moderate and difficult ones. Additionally, combining the feedback mechanism with human-designed step-by-step prompts (HDSP) improves performance for both simple and difficult samples with GPT-4. This validates that enhanced prompts with feedback are more effective in improving Text-to-SQL performance.

TABLE II: EX scores across different GPT model configurations.

LLMs		Simple	Moderate	Difficult
GPT-3	GPT-3	31.08	13.29	12.08
	GPT-3 + KG	45.44	26.14	19.01
GPT-3.5	GPT-3.5	34.70	13.33	11.11
	GPT-3.5 + KG	49.73	28.39	18.06
	GPT 3.5 + KG + HDSP	47.03	26.24	20.14
	GPT-3.5 + KG + Feedback	50.05	28.17	18.75
	GPT-3.5 + KG + Feedback + HDSP	48.68	32.18	20.83
GPT-4	GPT-4	36.96	24.64	12.32
	GPT-4 + KG	55.44	36.66	18.48
	GPT-4 + KG + HDSP	42.78	15.98	17.28
	GPT-4 + KG + Feedback	56.86	24.87	33.64
	GPT-4 + KG + Feedback + HDSP	62.86	26.83	35.68

TABLE III: VES scores across different GPT Model configurations

LLMs		Simple	Moderate	Difficult
GPT-3	GPT-3	36.20	15.43	14.42
	GPT-3 + KG	54.71	28.16	22.80
GPT-3.5	GPT-3.5	40.05	13.95	10.76
	GPT-3.5 + KG	51.83	28.91	16.72
	GPT 3.5 + KG + HDSP	49.14	27.43	15.84
	GPT-3.5 + KG + Feedback	51.98	28.83	17.08
	GPT-3.5 + KG + Feedback + HDSP	48.65	32.28	20.71
GPT-4	GPT-4	43.31	24.17	13.96
	GPT-4 + KG	60.31	34.76	20.08
	GPT-4 + KG + HDSP	61.71	25.63	31.55
	GPT-4 + KG + Feedback	60.43	24.10	31.68
	GPT-4 + KG + Feedback + HDSP	61.98	23.82	31.87

V. Related Work

Text-to-SQLs, which uses natural language queries to generate SQL statements, particularly through LLMs like GPT, is a rapidly evolving field. It aims to simplify the creation of SQL statements, enabling users to retrieve information using natural language. However, current performance of LLMs in Text-to-SQLs does not yet meet application requirements compared to human performance [4]. To advance Text-to-SQL, the BIRD benchmark [4] has been proposed to assess the readiness of LLMs as database interfaces through text-to-SQL tasks. BIRD features a comprehensive dataset across multiple domains to challenge models with real-world database values and the need for external knowledge in SQL statement generation.

Based on BIRD, several significant works have been completed to promote Text-to-SQLs. Sun *et. al* introduced the SQL-PaLM framework [19] through using few-shot prompting and instruction fine-tuning. In few-shot prompting, it explores the effectiveness of consistency decoding combined with execution-based error filtering. Instruction fine-tuning delves into understanding the critical paradigms that influence the performance of fine-tuned LLMs. Additionally, it proposes a test-time selection method to further refine accuracy by integrating SQL outputs from multiple paradigms with execution feedback as guidance. Pourreza *et. al* proposed DIN-SQL [12], which decomposes the Text-to-SQL task into four sub-steps: schema linking, complexity classification, SQL prediction, and self-correction. It also leverages few-shot prompting techniques. Experiments show that DIN-SQL not only narrows the performance gap between fine-tuned models and prompting approaches but also achieves state-of-the-art accuracy on the Spider and BIRD benchmarks. Li *et. al* proposed CodeS [20], a series of pre-trained language models designed specifically

for generating SQL statements from natural language inputs. Ranging from 1B to 15B parameters, CodeS models aimed to overcome the limitations of closed-source LLMs by being fully open-source and using an incremental pre-training approach on a curated SQL-centric corpus. CodeS enhances Text-to-SQL performance with comprehensive database prompts and a bi-directional data augmentation technique to facilitate adaptation to new domains. Wang *et. al* proposed a multi-agent system [17] to enhance SQL statement generation from natural language queries. It consists of three agents: the Decomposer, which breaks down complex questions; the Selector, which filters relevant database schema elements; and the Refiner, which corrects SQL errors. The framework leverages chain-of-thought reasoning and few-shot learning, significantly improving accuracy and robustness in SQL statement generation.

Although previous research has advanced LLM-based Text-to-SQL systems, these approaches often require complex data preprocessing or data augmentation techniques. In this study, we aim to enhance LLM-based Text-to-SQL by using straightforward feedback mechanisms and human-designed step-by-step prompts.

VI. Conclusion and Future Work

Recent advancements in LLM-based Text-to-SQL systems have improved the user experience for non-experts in retrieving information from relational databases using natural language queries. However, current state-of-the-art (SOTA) LLM-based Text-to-SQL models often require significant effort to develop auxiliary tools like SQL classifiers for optimal performance. This paper proposes a novel approach that enhances LLM-based Text-to-SQL performance using feedback based solely on SQL Quality Measurement. The proposed method undergoes comprehensive validation on the BIRD benchmark, assessing EX and VES across various Text-to-SQL difficulty levels. Experimental results reveal competitive performance in both EX and VES compared to SOTA models like GPT-4 and T5. Future work will involve validating the proposed method with more advanced LLMs and improving VES performance by reducing the complexity of feedback mechanisms.

Acknowledgment

This research work is supported by NASA under award number 80NSSC22KM0052, by the Army Research Office under cooperative agreement number W911NF-24-2-0133, and by the NSF under award number 2428761. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of NASA, NSF, or the Army Research Office or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

References

[1] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si et al., "A survey on text-to-sql parsing: Concepts, methods, and future directions," arXiv preprint arXiv:2208.13629, 2022.

[2] A. Kumar, P. Nagarkar, P. Nalhe, and S. Vijayakumar, "Deep learning driven natural languages text to sql query conversion: A survey," arXiv preprint arXiv:2208.04415, 2022.

[3] B. Zhang, Y. Ye, G. Du, X. Hu, Z. Li, S. Yang, C. H. Liu, R. Zhao, Z. Li, and H. Mao, "Benchmarking the text-to-sql capability of large language models: A comprehensive evaluation," arXiv preprint arXiv:2403.02951, 2024.

[4] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo et al., "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," Advances in Neural Information Processing Systems, vol. 36, 2024.

[5] R. Cai, J. Yuan, B. Xu, and Z. Hao, "Sadga: Structure-aware dual graph aggregation network for text-to-sql," Advances in Neural Information Processing Systems, vol. 34, 2021, pp. 7664–7676.

[6] D. Choi, M. C. Shin, E. Kim, and D. R. Shin, "Ryansql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases," Computational Linguistics, vol. 47, no. 2, 2021, pp. 309–332.

[7] B. Hui, R. Geng, L. Wang, B. Qin, Y. Li, B. Li, J. Sun, and Y. Li, "S2sql: Injecting syntax to question-schema interaction graph encoder for text-to-sql parsers," in Findings of the Association for Computational Linguistics: ACL 2022, 2022, pp. 1254–1262.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," arXiv preprint arXiv:1810.04805, 2018.

[9] X. V. Lin, R. Socher, and C. Xiong, "Bridging textual and tabular data for cross-domain text-to-sql semantic parsing," in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 4870–4888.

[10] J. Qi, J. Tang, Z. He, X. Wan, Y. Cheng, C. Zhou, X. Wang, Q. Zhang, and Z. Lin, "Rasat: Integrating relational structures into pretrained seq2seq model for text-to-sql," in Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, 2022, pp. 3215–3229.

[11] J. Li, B. Hui, R. Cheng, B. Qin, C. Ma, N. Huo, F. Huang, W. Du, L. Si, and Y. Li, "Graphix-t5: mixing pre-trained transformers with graph-aware layers for text-to-sql parsing," in Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence, 2023, pp. 13 076–13 084.

[12] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," Advances in Neural Information Processing Systems, vol. 36, 2024.

[13] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," arXiv preprint arXiv:2308.15363, 2023.

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou et al., "Chain-of-thought prompting elicits reasoning in large language models," Advances in neural information processing systems, vol. 35, 2022, pp. 24 824–24 837.

[15] H. Zhang, R. Cao, L. Chen, H. Xu, and K. Yu, "Act-sql: In-context learning for text-to-sql with automatically-generated chain-of-thought," in Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 3501–3532.

[16] M. Pourreza and D. Rafiei, "Dts-sql: Decomposed text-to-sql with small large language models," arXiv preprint arXiv:2402.01117, 2024.

[17] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, Q.-W. Zhang, Z. Yan, and Z. Li, "Mac-sql: Multi-agent collaboration for text-to-sql," arXiv preprint arXiv:2312.11242, 2023.

[18] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat et al., "Gpt-4 technical report," arXiv preprint arXiv:2303.08774, 2023.

[19] R. Sun, S. O. Arik, H. Nakhost, H. Dai, R. Sinha, P. Yin, and T. Pfister, "Sql-palm: Improved large language modeladaptation for text-to-sql," arXiv preprint arXiv:2306.00739, 2023.

[20] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "Codes: Towards building open-source language models for text-to-sql," arXiv preprint arXiv:2402.16347, 2024.